URP / □段性汇报

□向 GPU 张量程序反编译的

可信配对数据构建

URP □段性汇报 · 从 PtxDec 论文到 Agent 数据管线

PyTorch 任务 → 验证后的 CUDA → PTX / SASS

论文理解 | 系统实现 | 证据核查 | 下一步实验

□段定位：已实现上游数据管线，反编译效果待验证

为什么先解决数据问题?

研究目标: 从低层表示恢复可读、可执行的 CUDA

低层代码难以理解

- 编译优化把表达式、循环和访存拆成指令。
- 逐指令翻译容易丢失高层结构与调度意图。
- 可读 CUDA 有助于分析计算逻辑和性能实现。

训练与评估需要成套证据

- 输入: 编译后的 PTX, SASS 可作拓展。
- 标签: 可读 CUDA kernel, 不要求逐字还原原源码。
- 验证: 输入、参数、launch 配置和参考输出。

本□段问题: 能否自动生产可验证、可追溯的 CUDA–低层表示配对数据?

PtxDec：数据、表示与训练协同

Enhancing LLM to Decompile Optimized PTX to Readable CUDA for Tensor Programs · ASE 2025

① 编译器数据增强

Ansor：调度多样性

Welder：子图多样性

→

② 数据后处理

CUDA 重构

MinHash / LSH 去重

→

③ 模型增强

PTX → Rolled-PTX

LoRA / SFT → CUDA

40 万对 CUDA–PTX kernel

- 保存 shape、grid / block 等测试配置。
- 同一计算的不同调度实现构成重要数据来源。

Rolled-PTX 的作用

- 用启发式模式匹配压缩展开的循环。
- 论文报告 PTX 平均长度减少 41%。
- 它是论文自定义表示，非标准可直接执行 PTX。

启发：数据价值不仅在数量，还在调度多样性、可读标签与可重建测试环境。

论文 §III, PDF pp.4–7; §IV-F, p.10 (Fig.14)

论文结果说明什么?

先区分编译通过、运行成功和输出正确

□型 / 设置	编译率	运行正确率	I/O 正确率
Qwen2.5-Coder-7B	42.16%	34.61%	29.26%
PtxDec-7B	**97.24%**	**96.40%**	**90.89%**
移除 CUDA 重构	75.07%	70.15%	59.49%
移除 PTX 重卷	89.05%	72.17%	52.57%

域外任务更有挑战

手工优化 kernel 上, PtxDec-7B / 32B 的 I/O 正确率为 **29.10% / 60.31%**。

对本课题的启示

采用分□段正确性指标，并独立检查可读性与调度保留；论文数值不能直接作为 SASS 基线。

论文 Tables I–IV, PDF pp.8–10; 主测试集 4,000 kernels

我的工作：补充一条 Agent 数据路线

工程进展与研究假设分别表述

维度	PtxDec 论文路线	本阶段路线
CUDA 来源	深度学习编译器生成	Agent 实现 Ops-6K 的 PyTorch 任务
多样性机制	子图搜索+调度搜索	算子组合；调度多样性尚未量化
低层表示	PTX / Rolled-PTX	已导出 PTX、cubin、SASS
模型工作	反编译模型微调与评估	上游生成、修复、验证和归档

待检验假设：Agent 数据能否补充编译器数据的覆盖范围，并形成有效的反编译测试集?

本地 cuda_agent/workflow.py、docs/WORKFLOW.md; 论文 §III

已实现：生成—验证—修复—归档

远程模型产生代码，本地工具执行与裁决

任务与契约

model.py

shape / dtype / stride

固定参数与检查点

→

三阶段生成

Python 接口

CUDA kernel / launcher

C++ 绑定

→

本地评估

源码 / 接口检查

编译

5 次 seed 输出验证

失败证据+检查点+候选历史 → 修复代码 → 重新评估 ↺

通过后导出

- 性能测量用于记录，不设置加速比门槛。
- 生成 PTX / cubin / SASS，归档源码和日志。

当前修复预算

- 初始生成一次，最多成功生成 3 次修复。
- 最多评估 4 个候选；退化时恢复最好候选。
- 修复 API 连续失败 3 次则终止。

cuda_agent/workflow.py:48,120,444–512; 历史运行版本与当前实现有差异

可靠性来自执行证据，而非模型自评

契约帮助定位问题，正式验证决定候选是否通过

① 契约与接口

- 保留模型构造参数、forward 接口与 state_dict。
- 采集 shape / dtype / stride 和中间检查点。
- 检查点诊断辅助定位首次偏差。

② 正式验证

- 使用相同初始化参数和输入比较参考与候选。
- 执行 5 次不同 seed 的正常 forward 验证。
- 检查输出结构、类型、形状及数值误差。

③ 候选与证据

- 保留源码快照、清单哈希、日志和失败输入。
- 编译失败禁止验证陈旧的二进制。
- 修复后重新检查并评估。

④ 可信边界

- 有限测试只能支持“已测用例通过”。
- 不能据此证明所有输入下的语义等价。
- 仍需边界形状、特殊数值和 launch 覆盖测试。

阶段成果：先报告可复核的事实

基于当前本地文件核查 · 统计范围 task 0000–0049

21
已有任务归档目录

20
含 CUDA / PTX / cubin / SASS

18
完整产物+验证通过记录

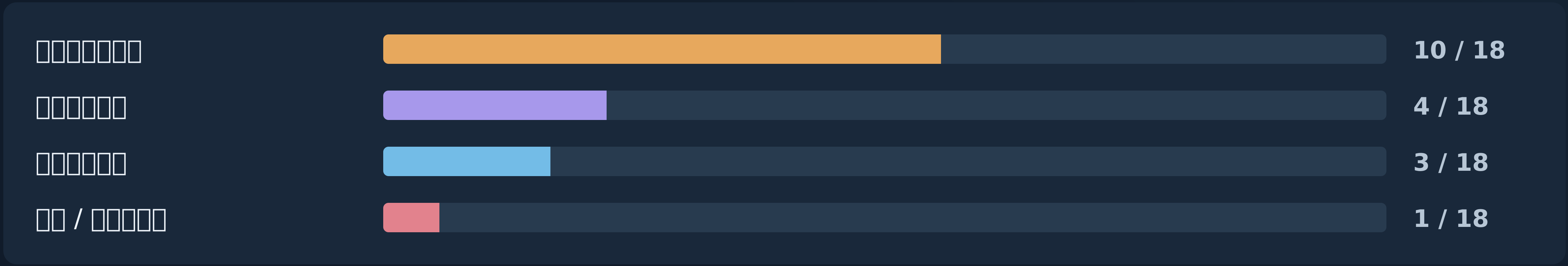
最新已完成报告：**35** 份

17 成功 / **18** 失败，成功占已完成报告的 **48.6%**。另外 15 个任务没有完成报告，不能作为失败或成功归类。

历史归档与最新运行分开计数

归档可保留早期成功结果，最新重跑可能失败。18 个可复核历史归档任务与 17 个最新成功任务是不同口径。

18 是任务级归档数量，尚未完成 kernel 级配对审计；原稿 32/50 暂无可复核依据。



失败复盘：定位到具体阶段

分母统一：18 个最新已完成失败报告

验证失败不全是数值误差

最新报告涉及绑定参数不匹配、shape 偏差、返回结构不符等；先拆分失败类型，再决定修复策略。

预检失败需区分根因

task 31 / 36 含导入□段 KeyboardInterrupt; 32 / 38 为参考执行异常，不能统称“算子兼容性”。

优先降低接口与输出结构错误，再补充数值边界；新旧版本对比需固定任务、模型与预算。

最新报告: task 0,1,2,3,8,10,13,14,19,20,21,24,27,29,31,32,36,38

真实样例：Task 0006 的 ge kernel

ReflectionPad2d → ge → remainder · 同一归档中的 CUDA 与 SASS 节选

CUDA · 保留原始声明与计算语句

```
__global__ void ge_scalar_kernel(
    const float* __restrict__ input,
    unsigned char* __restrict__ output,
    float scalar,
    int total_elements,
    int in_stride_n, int in_stride_c, int in_stride_h, int
in_stride_w,
    int H, int W, int C) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    int stride = blockDim.x * gridDim.x;
    for (int i = idx; i < total_elements; i += stride) {
        int w = i % W;
        int tmp = i / W;
        int h = tmp % H;
        tmp /= H;
        int c = tmp % C;
        int n = tmp / C;
        const float* in_ptr = input + n * in_stride_n + c *
in_stride_c + h * in_stride_h + w * in_stride_w;
        float val = *in_ptr;
        output[i] = (val >= scalar) ? 1 : 0;
    }
}
```

SASS · cuobjdump 真实输出 / sm_89

```
/*0000*/                   IMAD.MOV.U32 R1, RZ, RZ, c[0x0][0x28] ;
/*0010*/                   S2R R3, SR_CTAID.X ;
/*0020*/                   S2R R0, SR_TID.X ;
/*0030*/                   IMAD R3, R3, c[0x0][0x0], R0 ;
/*0040*/                   ISETP.GE.AND P0, PT, R3, c[0x0][0x174],
PT ;
/*0050*/               @P0 EXIT ;
/*0060*/                   IMAD.MOV.U32 R0, RZ, RZ, c[0x0][0x0] ;
/*0070*/                   ULDC.64 UR4, c[0x0][0x118] ;
/*0080*/                   BSSY B0, 0x8f0 ;
/*0090*/                   IMAD R0, R0, c[0x0][0xc], RZ ;
/*00a0*/                   I2F.U32.RP R6, R0 ;
/*00b0*/                   IMAD.MOV R9, RZ, RZ, -R0 ;
```

一份 custom_ops.cu 可以含多个 kernel：必须按函数符号配对，不能视为“一条 CUDA 对应一条 SASS”。

历史最佳候选：0002

右侧仅展示函数入口指令，非完整 kernel。

results/task_0006/kernels/custom_ops.cu:51; custom_ops.sass:664; verification.log

从任务归档到反编译数据集，还差什么?

下一步优先完善样本定义与配对审计

任务级归档

多 kernel 源码

Python 包装与测试日志

→

kernel 级抽取

符号与源码位置映射

CUDA ↔ PTX ↔ SASS

→

可执行数据样本

参数 / ABI / launch 配置

测试输入与参考输出

配对与复现字段

- 记录源码 / 产物哈希、编译命令与工具链。
- 核查每个 kernel 是否存在对应 .entry / Function。
- 区分外部库调用与可独立恢复的计算。

评估集先于微调准备

- 先按任务来源和近重复簇划分数据集。
- 同一 kernel 的架构、优化变体必须在同一划分。
- AST / token 相似度阈值需校准，不能直接套 0.9。

正向导出: CUDA 分别经 nvcc -ptx 与 nvcc -cubin; 再从 cubin 提取 SASS。

下一阶段实验：先做 PTX 对照，再探索 SASS

所有反编译结果均为计划，当前尚无测得通过率

问题	实验控制	报告指标
低层表示增加多少难度?	同一 kernel、模型、提示词与生成预算 分别输入 PTX / SASS	编译率、运行率、I/O 通过率
契约信息是否有帮助?	仅低层代码 vs +接口 / launch 元数据 不提供参考源码和算子实现	通过率差异、失败类型
Agent 修复是否有效?	固定任务与初始候选 有 / 无修复，统计调用开销	新增通过数、耗时和 token 成本
数据是否保留实现特征?	只对 I/O 通过的反编译结果分析	可读性、延迟与访存结构偏差

低 zero-shot 通过率只能说明任务困难；数据价值需要受控对比与训练增益支持。

实验设计，尚未执行；评价框架参考论文 §IV-A

下一步交付与验收

先把证据闭环，再扩大规模

P0 完成 **18** 个归档任务的 **kernel** 级审计

交付：配对清单、符号映射、哈希和可重放测试；列出不能独立评估的样本。

P0 建立 **PTX / SASS zero-shot** 基线

交付：固定评估集与三□段通过率，逐例记录失败和输入元数据。

P1 做修复消融与覆盖扩展

交付：同预算对照；扩大到 200+ 任务为目标，并单独统计 kernel 数量。

P2 再开展模型微调与调度分析

前提：完成数据划分和质量审计；交付：训练增益、域外结果与可读性分析。

本次希望讨论：是否优先以**PTX** 建立可比较基线，再逐步扩展到 SASS？

备用页：统计口径与来源

供追问时定位证据

本地核查规则

- 归档：results/task_* 共 21 个目录。
- 完整：至少一份非空 .cu / .ptx / .cubin / .sass。
- 验证：verification.log 包含 [PASS] verify success。
- 最新报告：每个任务选择时间戳最大的已完成报告。

需要谨慎解释的差异

- 历史归档 18 个 ≠ 最新成功 17 个。
- 没有完成报告的 15 个任务状态未知。
- task 9 有验证记录，但缺 PTX / cubin / SASS。
- task 0 / 2 有低层产物，但未见通过标记。

论文：Sun et al., ASE 2025, DOI 10.1109/ASE63991.2025.00185

本地依据：docs/urp_report/evidence.json、核查说明.md、results/、logs/。

历史代码与当前实现有差异；未执行新的 GPU / 模型实验。